SAP-1 Computer Architecture

Detailed Study Notes

---

1. Introduction to SAP Computers

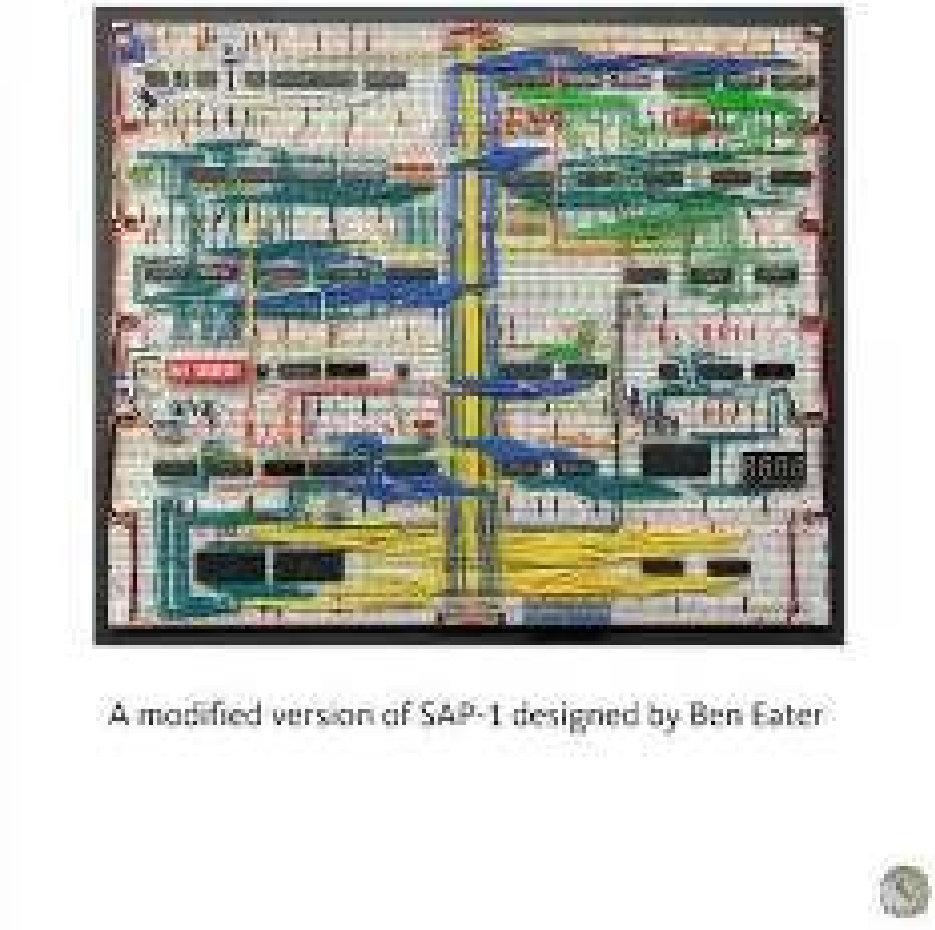


Fig 1 — SAP-1 introduction slide (a modified version built by Ben Eater)

SAP stands for **Simple-As-Possible**.

SAP computers (SAP-1, SAP-2, SAP-3) were first introduced by **Albert Paul Malvino** in his book **"Digital Computer Electronics"**.

- SAPs have a very simplified computer architecture.
- They help us understand the crucial ideas behind computer operation without unnecessary complexity.
- SAP-1 is the first evolution of SAPs.

2. Architecture of SAP-1

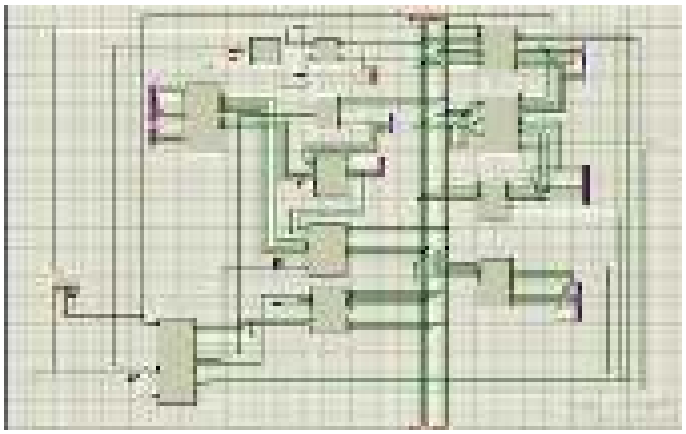


Fig — Full SAP-1 software simulation (circuit simulator)

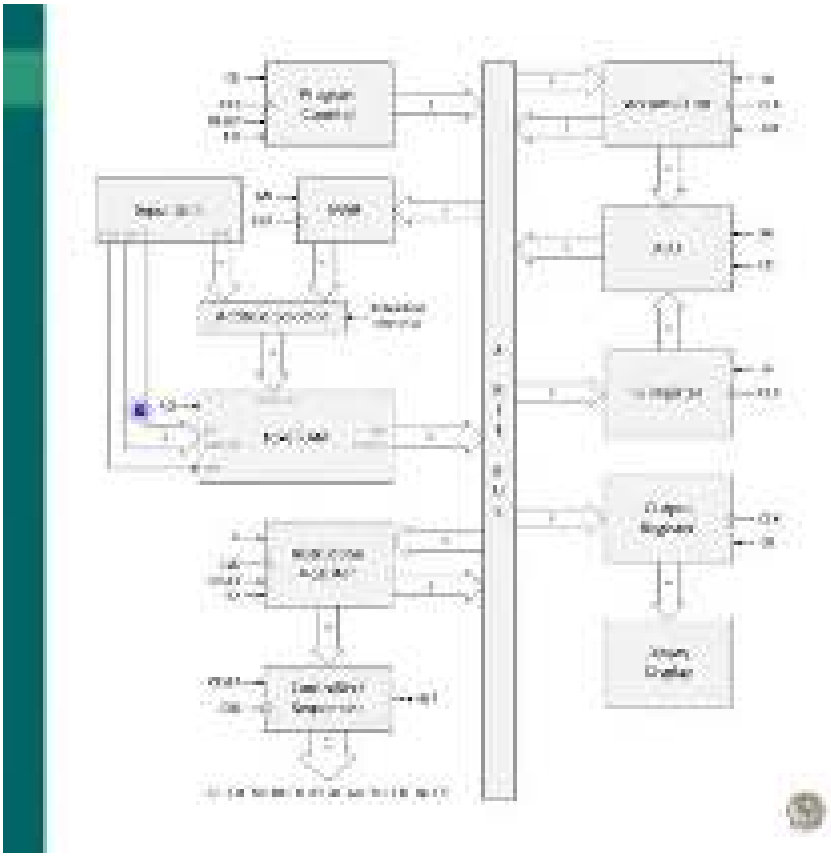
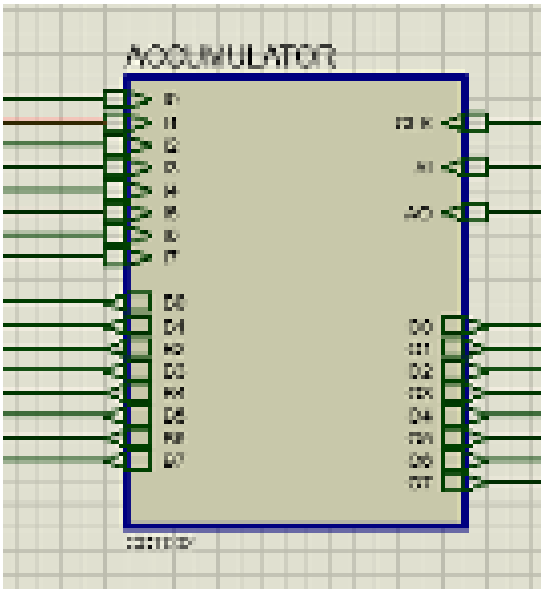


Fig 2a — SAP-1 architecture overview: hardware components

Hardware Components

- 8-bit bus
- 16×8 bit RAM
- A 4-bit Program Counter (PC)
- Input Unit and Memory Address Register (MAR)
- 8-bit Arithmetic and Logic Unit (ALU)
- Two 8-bit registers: Accumulator (Register A) and Register B



Software simulation — Accumulator circuit

- A controller/sequencer for controlling all other blocks
- Output unit for showing output in binary form

Instruction Set (5 Instructions)

SAP-1 executes exactly **5 instructions**. Each instruction is 8 bits: the upper 4 bits are the **opcode**, the lower 4 bits are the **address/operand**.

Mnemonic	Opcode (4-bit)	Full Name	Description
LDA	0000	Load A	Loads data from a specified RAM address into the Accumulator.
ADD	0001	Add	Adds data from a specified RAM address to the Accumulator. Result stored in Accumulator.
SUB	0010	Subtract	Subtracts data from a specified RAM address from the Accumulator. Result stored in Accumulator.
OUT	0011	Output	Loads data from the Accumulator and sends it to

			the Output Register for display.
HLT	1111	Halt	Halts the computer by stopping the main clock.

Clock Modes (2 Modes)

The main clock signal driving SAP-1 has two modes of operation:

Mode	Description
Auto	The clock oscillates automatically at a fixed frequency. The computer steps through instructions continuously without manual intervention.
Manual	Each clock pulse is triggered manually (e.g., by pressing a button). Useful for debugging and step-by-step observation of instruction execution.

3. The 8-bit Bus

The 8-bit bus consists of 8 wires/lines used for data transfer between all modules.

Bus Rules

Rule i: At any single time instance, only one module will send data to the bus, and only one module will receive that data.

Rule ii: Two modules can never send data to the bus at the same time — this causes **bus contention**.

Rule iii: When a module is sending data to the bus, all other modules' outputs must be disconnected. This is achieved using **tri-state buffers**.

Tri-State Buffer — 3 States

A tri-state buffer has **three possible output states**. It is the key component that allows modules to connect to and disconnect from the shared bus without causing contention.

State Name	Enable (En)	Output	Meaning
Logic HIGH	**1**	Out = 1 (HIGH)	En=1 allows input through. Module actively drives bus HIGH.
Logic LOW	**1**	Out = 0 (LOW)	En=1 allows input through. Module actively drives bus LOW.
High-Z (Disconnected)	**0**	Floating (Hi-Z)	En=0 disconnects output entirely — open circuit. Module is fully isolated from the bus. This is the key state that prevents bus contention.

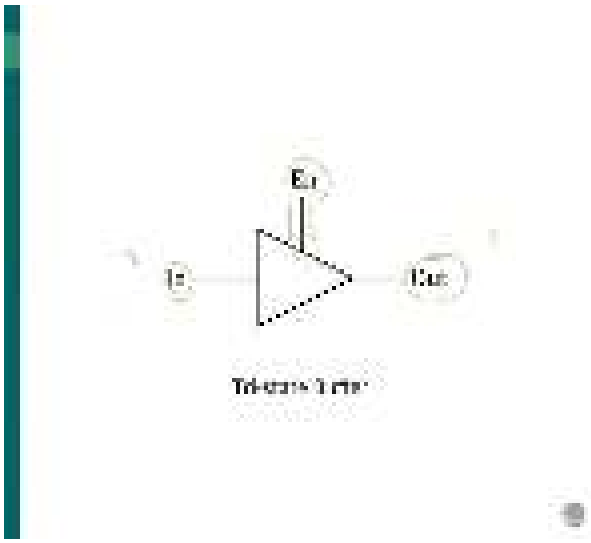
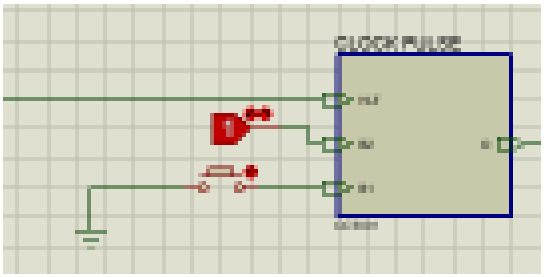


Fig 3a — Tri-state buffer symbol

4. Program Counter (PC)



Software simulation — Clock Pulse circuit

Fig 4 — Program Counter module and its connections

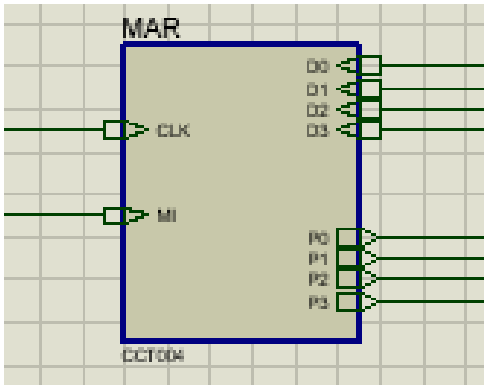
The Program Counter is a **4-bit counter** counting from **0000** to **1111**.

- Stores and sends out the memory address of the next instruction to be fetched.
- Increments its value after each instruction cycle.

Control Bits

Control Signal	Abbr.	Description
Counter Enable	**CE**	Increments the program counter on the next clock cycle.
Counter Out	**CO**	Puts the current program counter value on the bus. Output is disconnected when CO is low.

5. Memory Address Register (MAR)



Software simulation — MAR circuit

Fig 5 — MAR module and its connections to Address Selector and RAM

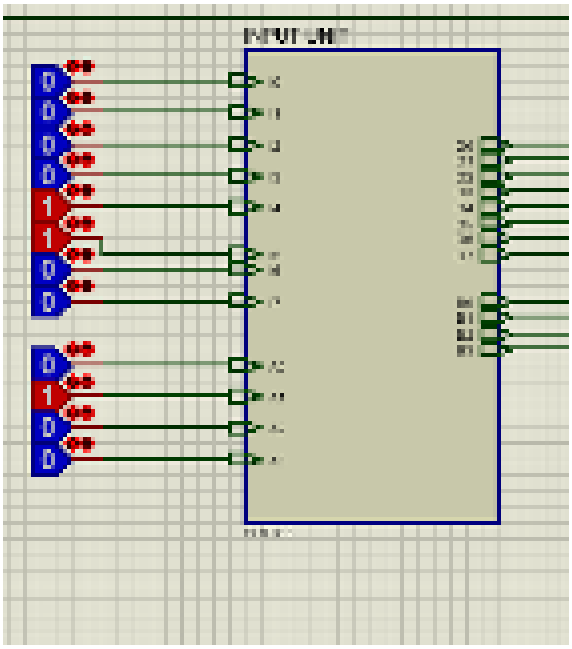
The MAR is a 4-bit register.

- Stores the memory address of RAM from which data will be fetched.
- This 4-bit address is obtained via the bus from the Program Counter and then stored.
- The stored address is sent to the RAM where data or instructions are read from.

Control Bits

Control Signal	Abbr.	Description
Memory Address In	**MI**	Stores the current values of the bus into the Memory Address Register (MAR).

6. Input Unit



Software simulation — Input Unit circuit

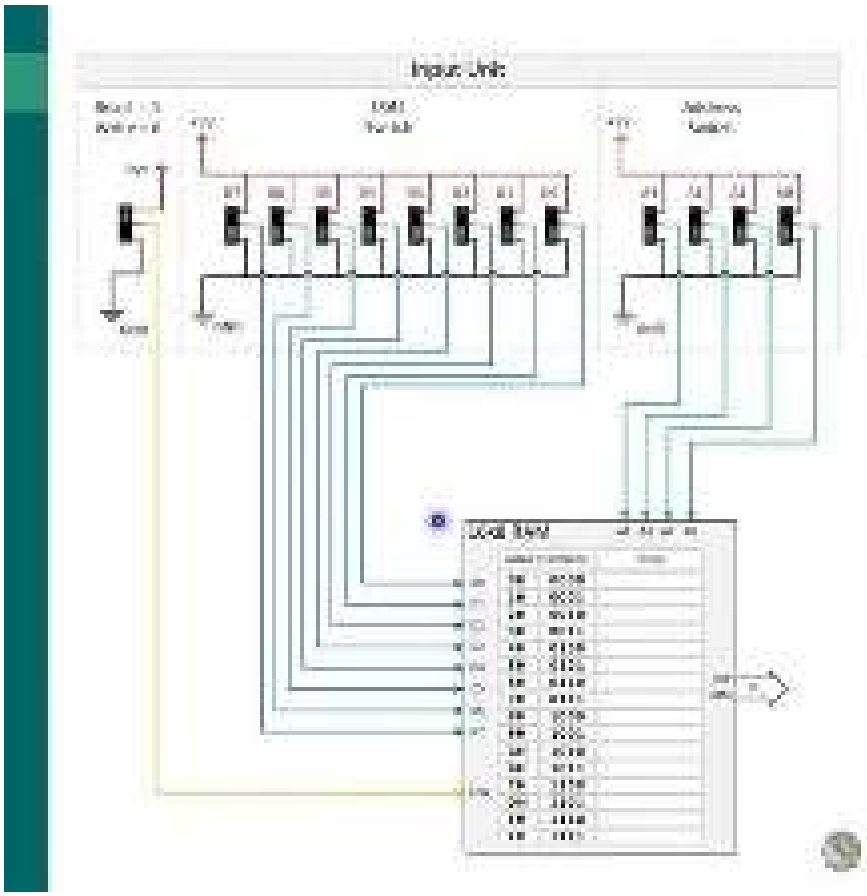


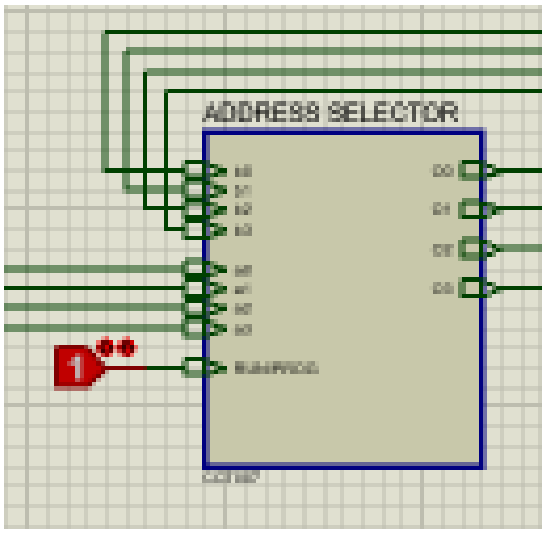
Fig 6 — Input Unit switch layout and connection to RAM

The Input Unit is used when the computer needs to be programmed by writing data into the RAM — the **PROG state**.

Switches

Switch	Description
DATA Switch	8 switches (D7–D0) for writing 8-bit data to RAM.
ADRS Switch	4 switches (A3–A0) for accessing a particular RAM address for writing.
R/W Switch	Switches between read (1) and write (0) mode.

7. RUN/PROG Selector Switch



Software simulation — Address Selector circuit

The computer has two operating states:

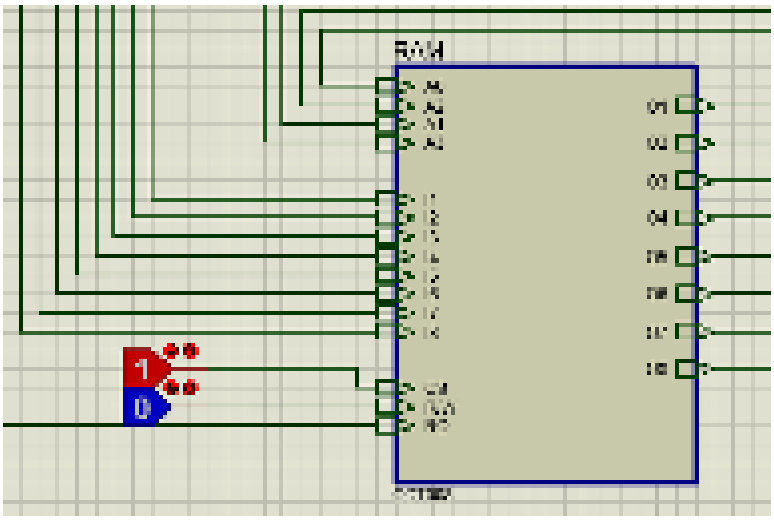
RUN: When a program is in execution on the computer.

PROG: Before running, the computer must be programmed by writing data into the RAM.

Based on the RUN/PROG state, a 4-bit address is selected from either **MAR** or **Input Unit** using the **Address Selector**.

State	Address Source Selected
RUN	MAR will be selected
PROG	Input Unit will be selected

8. Random Access Memory (RAM)



Software simulation — 16×8 RAM circuit

RAM size: **16×8 bit** — 16 memory locations, each storing 8 bits.

- The RAM is programmed using the address switches and data switches of the input unit.
- During a computer run, the RAM receives its 4-bit address from the MAR.

Control Bits

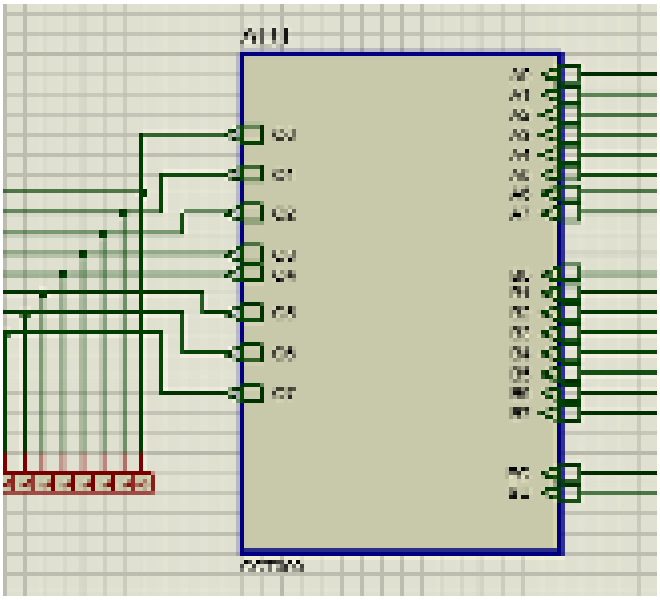
Control Signal	Abbr.	Description
RAM Out	**RO**	Puts the currently selected RAM byte onto the bus. Output is disconnected when RO is low.

Writing to RAM — Example

Write 8-bit data **0110 1100** at address **1001**:

- **D7–D0 DATA switches:** set to 0110 1100
- **A3–A0 ADRS switches:** set to 1001
- **R/W switch:** set to Write (0)
- Toggle write cycle to store data at address 9H.

9. Arithmetic and Logic Unit (ALU)



Software simulation — ALU circuit

The ALU performs 8-bit binary addition and subtraction.

Control Bits

Control Signal	Abbr.	Description
Addition/Subtraction Select	**SU**	SU=0: addition operation | SU=1: subtraction operation
ALU Out	**EO**	Puts the ALU output onto the bus. Output is disconnected when EO is low.

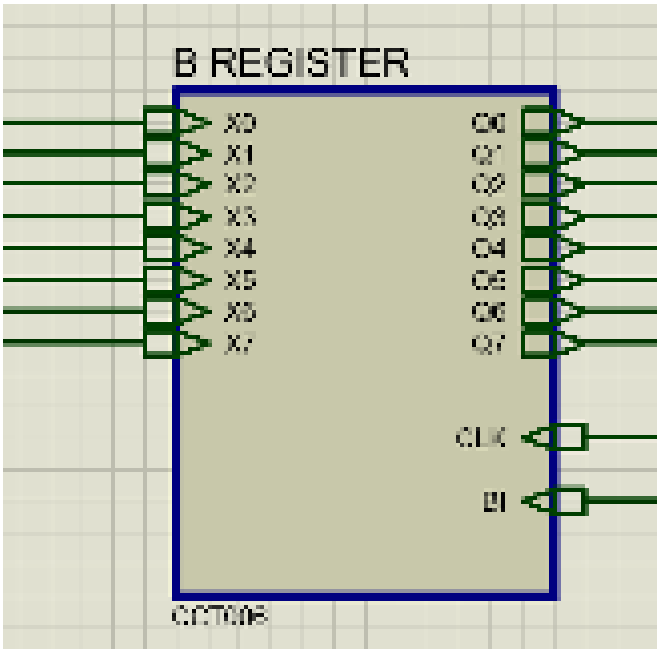
Accumulator (Register A)

The Accumulator is an 8-bit register that:

- Stores one operand for ALU operations.
- Receives and holds the result after ALU computation.
- Its output is always connected internally to the ALU (permanent connection, independent of bus).

Control Signal	Abbr.	Description
Accumulator In	**AI**	Loads the current value on the bus into the Accumulator on the next clock edge.
Accumulator Out	**AO**	Puts the Accumulator value onto the 8-bit bus. Output is disconnected when AO is low (tri-state buffer).

B Register



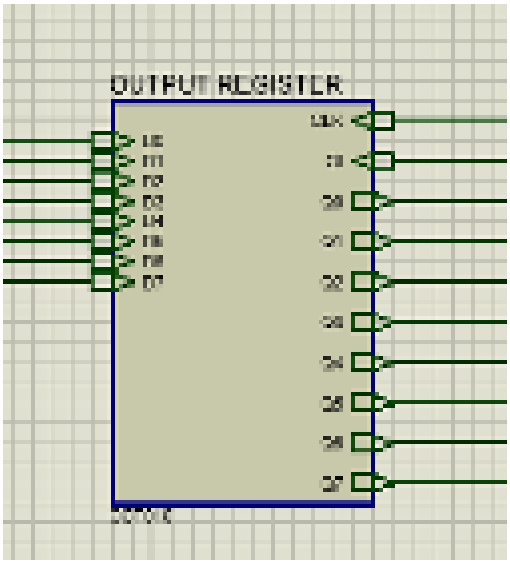
Software simulation — B Register circuit

The B Register is an 8-bit register that:

- Stores the second operand for ALU operations.
- Its output is always fed directly into the ALU (permanent internal connection, not via bus).
- Unlike the Accumulator, the B Register has no output to the bus — it only receives data.

Control Signal	Abbr.	Description
B Register In	**BI**	Loads the current value on the bus into the B Register. This is then immediately available to the ALU as the second operand.

10. Output Unit



Software simulation — Output Register circuit

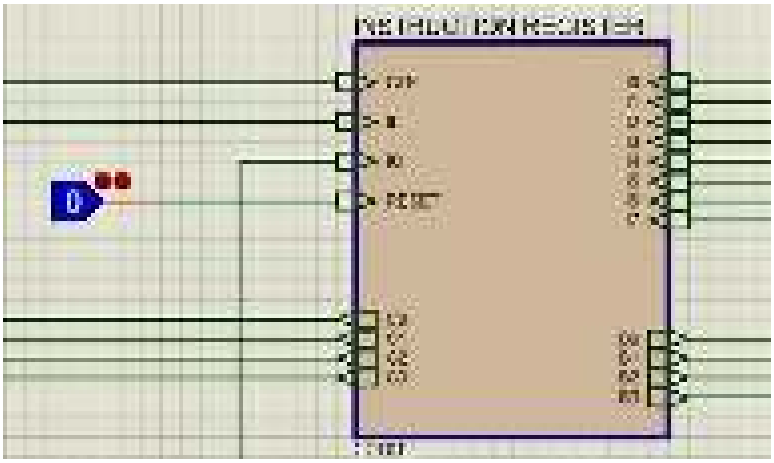
The Output Unit is an 8-bit register that:

- Stores the result to display when requested.
- Receives data from the Accumulator via the bus when the OUT instruction is executed.
- Connects to a Binary Display (LEDs) for showing output

Control Bits

Control Signal	Abbr.	Description
Output Register In	**OI**	Stores the current values of the bus into the Output Register.

11.1 Instruction: SUB (Subtract)



Software simulation — Instruction Register circuit

Instruction	Subtract
Mnemonic	**SUB**
Opcode	**0010**

Function: Subtracts the value stored at a particular RAM address from the value currently in the Accumulator. The RAM address is given by the address field (lower 4 bits) of the instruction.

Instruction format (8 bits): **0010 AAAA** — where AAAA is the 4-bit RAM address

Example	
Assembly	**SUB BH**
In Binary	**0010 1011**

Explanation: Opcode **0010** = SUB. Address **1011** = BH (address 11 in hex). The computer subtracts the byte stored at RAM address BH from the Accumulator and stores the result back in the Accumulator.

11.2 Instruction: OUT (Output)

Instruction	Output
Mnemonic	**OUT**
Opcode	**0011**

Function: Loads data from the Accumulator and sends it to the Output Register. The Output Register then drives the Binary Display.

Instruction format (8 bits): **0011** 0000 — address field is unused (set to 0000)

Example	
Assembly	**OUT**
In Binary	**0011 0000**

Explanation: Opcode **0011** = OUT. No address field is needed. The current value of the Accumulator is transferred to the Output Register and shown on the binary display.

11.3 Instruction: HLT (Halt)

Fig 11.3 — HLT instruction slide

Instruction	Halt
Mnemonic	**HLT**
Opcode	**1111**

Function: Halts the process of the computer by stopping the main clock. All execution ceases immediately.

Instruction format (8 bits): **1111** 0000 — address field is unused

Example	
Assembly	**HLT**
In Binary	**1111 0000**

Explanation: Opcode **1111** = HLT. The Controller/Sequencer detects this opcode and stops the clock, freezing the computer in its current state.

12. Complete Control Signals Summary

Module	Signal	Full Name	Function
Program Counter	**CE**	Counter Enable	Increment PC on next clock cycle
Program Counter	**CO**	Counter Out	Put PC value on bus
MAR	**MI**	Memory Address In	Load bus value into MAR
RAM	**RO**	RAM Out	Put RAM byte on bus
Instruction Reg.	**II**	Instruction In	Load bus value into IR
Instruction Reg.	**IO**	Instruction Out	Put lower 4 bits of IR on bus
Accumulator	**AI**	Accumulator In	Load bus value into Accumulator
Accumulator	**AO**	Accumulator Out	Put Accumulator value on bus

ALU	**SU**	Subtract Select	0 = Add, 1 = Subtract
ALU	**EO**	Enable Out	Put ALU result on bus
B Register	**BI**	B Register In	Load bus value into B Register
Output Register	**OI**	Output In	Load bus value into Output Register

13. Instruction Cycle and T States

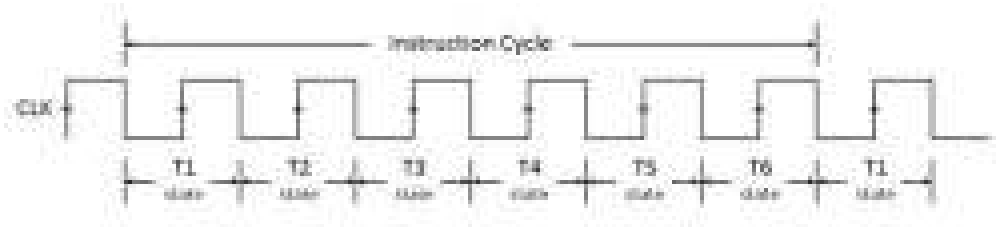


Fig 13 — One instruction cycle = 6 T states (T1–T6), each one clock pulse wide

What is an Instruction Cycle?

The **instruction cycle** is the total time required to fetch and execute one single instruction. In SAP-1, every instruction cycle consists of exactly **6 T states**.

What is a T State?

A **T state** (time state) is the time required to complete one **microinstruction** — a single atomic operation (e.g. put PC on bus, load MAR). Each T state lasts exactly **one clock pulse**.

Structure of an Instruction Cycle

An instruction cycle is divided into two sub-cycles:

Sub-cycle	T States	Purpose
Fetch Cycle	**T1, T2, T3**	Fetches the next instruction from RAM into the Instruction Register. Identical for every instruction.
Execution Cycle	**T4, T5, T6**	Executes the fetched instruction. Varies depending on which instruction is being executed.

Key insight: Because the fetch cycle is always the same, T1–T3 microinstructions are identical for every instruction in SAP-1. Only T4–T6 differ between instructions.

14. Fetch and Execution Cycles

14.1 Fetch Cycle (T1, T2, T3)

The fetch cycle is **identical for all instructions**. It retrieves the instruction stored at the address currently held in the Program Counter and loads it into the Instruction Register.

T State	State Name	Activated Control Bits
T1	Address State	**CO MI**
T2	Increment State	**CE**
T3	Memory State	**RO II**

Step-by-Step: What Happens in Each Fetch State

T1 — Address State (CO, MI): The Program Counter outputs its current value onto the bus (CO active). The Memory Address Register loads this value from the bus (MI active). Now the MAR holds the address of the next instruction.

T2 — Increment State (CE): The Program Counter increments by 1 (CE active), so it is ready to point to the next instruction for the following cycle. The bus is not used in this step.

T3 — Memory State (RO, II): The RAM outputs the byte stored at the address in MAR onto the bus (RO active). The Instruction Register loads this byte from the bus (II active). The instruction is now inside the IR, ready to be decoded and executed.

14.2 Execution Cycle (T4, T5, T6)

During the execution cycle the **Controller/Sequencer** reads the opcode from the Instruction Register and activates the appropriate control bits for T4, T5, and T6. These **vary per instruction**.

Execution Cycle — LDA (Load Accumulator)

T State	Activated Control Bits
T4	**IO MI**
T5	**AI RO**
T6	No operation

T4 (IO, MI): The lower 4 bits of the Instruction Register (the address field) are put on the bus (IO active). The MAR loads this address (MI active).

T5 (AI, RO): The RAM outputs the byte at the address now in MAR (RO active). The Accumulator loads this byte (AI active). The load is complete.

T6: No operation — LDA only needs 2 execution steps.

Execution Cycle — ADD

ADD:

T State	Activated Control Bits
T4	**IO MI**
T5	**RO BI**
T6	**EO AI**

T4 (IO, MI): Address field of instruction → bus → MAR.

T5 (RO, BI): RAM byte at that address → bus → B Register (second ALU operand).

T6 (EO, AI): ALU computes Accumulator + B Register (SU=0 so addition). Result → bus → Accumulator.

Execution Cycle — SUB

T State	Activated Control Bits
T4	**IO MI**
T5	**RO BI**

T6	**EO AI SU**

Identical to ADD for T4 and T5. At T6, **SU is also activated** alongside EO and AI, which tells the ALU to subtract instead of add (Accumulator − B Register). Result stored back in Accumulator.

Execution Cycle — OUT

T State	Activated Control Bits
T4	**AO OI**
T5	No operation
T6	No operation

T4 (AO, OI): Accumulator value → bus → Output Register. The Binary Display immediately reflects the new value. Only one step needed.

Execution Cycle — HLT

HLT does not require any execution cycle states. No registers are involved. The Controller/Sequencer simply activates the **HLT bit**, which stops the main clock immediately. The computer freezes in its current state.

14.3 Complete T-State Summary — All Instructions

Instr.	T1	T2	T3	T4	T5
LDA	CO MI	CE	RO II	**IO MI**	**AI RO / —**
ADD	CO MI	CE	RO II	**IO MI**	**RO BI / EO AI**
SUB	CO MI	CE	RO II	**IO MI**	**RO BI / EO AI SU**
OUT	CO MI	CE	RO II	**AO OI**	**— / —**
HLT	CO MI	CE	RO II	**HLT**	**— / —**

Note: T1–T3 (grey) are the **fetch cycle** — same for all instructions. T4–T6 (blue) are the **execution cycle** — unique per instruction.

15. Example Program

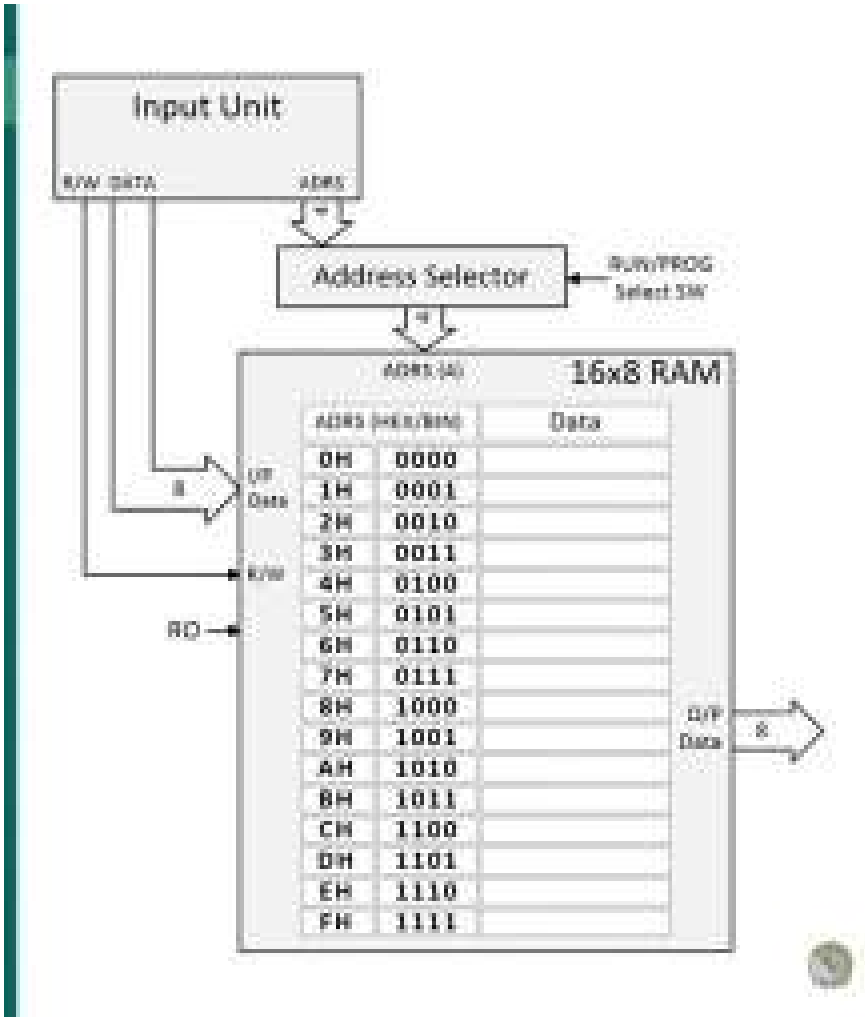


Fig 15a — Example program loaded into RAM (mnemonic form)

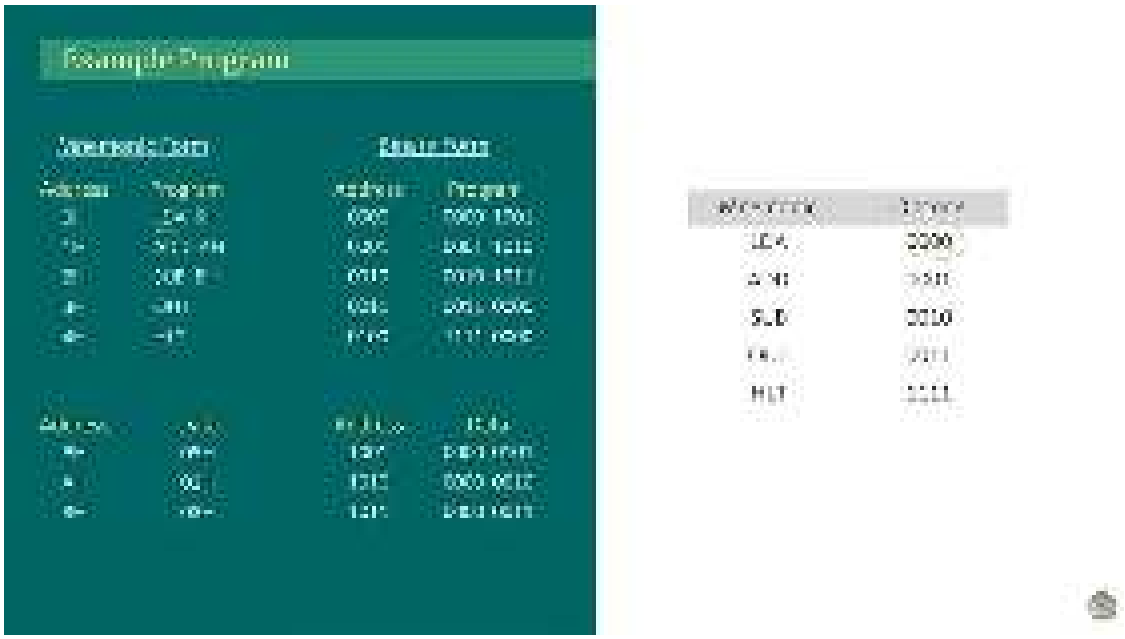


Fig 15b — Mnemonic form, binary form, and opcode reference

Program Description

This example program loads two values from memory, adds and subtracts them, outputs the result, then halts.

Program in Mnemonic Form

Address	Instruction
0H (0000)	LDA 9H — Load value at address 9H into Accumulator
1H (0001)	ADD AH — Add value at address AH to Accumulator
2H (0010)	SUB BH — Subtract value at address BH from Accumulator
3H (0011)	OUT — Output Accumulator value to display
4H (0100)	HLT — Halt the computer

Data Stored in Higher Memory

Address (Hex)	Address (Binary)	Data (Hex)	Data (Binary)
9H	**1001**	**05H**	0000 0101
AH	**1010**	**02H**	0000 0010
BH	**1011**	**03H**	0000 0011

Program in Binary Form

Address (Binary)	Instruction (Binary)	Breakdown
0000	**0000 1001**	Opcode 0000 (LDA) + Address 1001 (9H)
0001	**0001 1010**	Opcode 0001 (ADD) + Address 1010 (AH)
0010	**0010 1011**	Opcode 0010 (SUB) + Address 1011 (BH)
0011	**0011 0000**	Opcode 0011 (OUT) + Unused 0000
0100	**1111 0000**	Opcode 1111 (HLT) + Unused 0000

Step-by-Step Execution Trace

What the computer actually computes when this program runs:

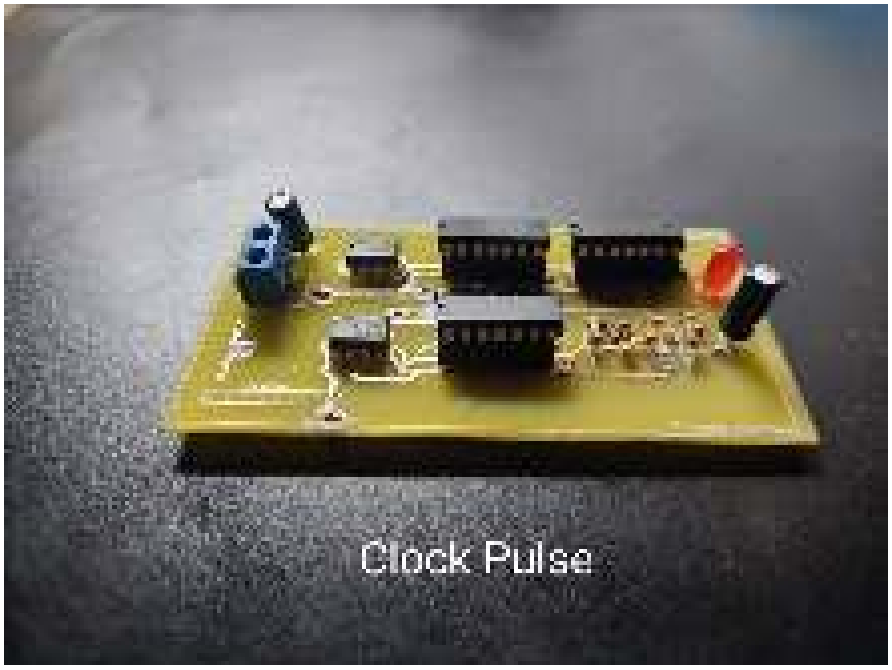
- **LDA 9H:** Accumulator ← 05H (load 5)
- **ADD AH:** Accumulator ← 05H + 02H = 07H (add 2, result = 7)
- **SUB BH:** Accumulator ← 07H − 03H = 04H (subtract 3, result = 4)
- **OUT:** Output Register ← 04H (display: 0000 0100 in binary)
- **HLT:** Clock stops. Final displayed value = 4 (decimal)

16. Hardware Implementation (PCB)

The following photographs show the physical PCB implementations of each SAP-1 module, designed and assembled by Team SAPiens at the Islamic University of Technology (IUT). Each module was built as a separate PCB and interconnected via the shared 8-bit bus.

Manufactured by Taru PCB. Estimated cost: PCB ~4,000 BDT | Components ~9,500 BDT | Miscellaneous ~3,000 BDT | Total ~16,500 BDT

16.1 Clock Pulse



PCB — Clock Pulse

555 Timer IC in Astable mode (auto) and Monostable mode (manual push-button). Drives the master clock line for all modules.

16.2 Program Counter



PCB — Program Counter

4-bit synchronous up-counter (0000→1111) built with 4 JK Flip-Flops. CE increments on each clock cycle; CO drives the address onto the bus.

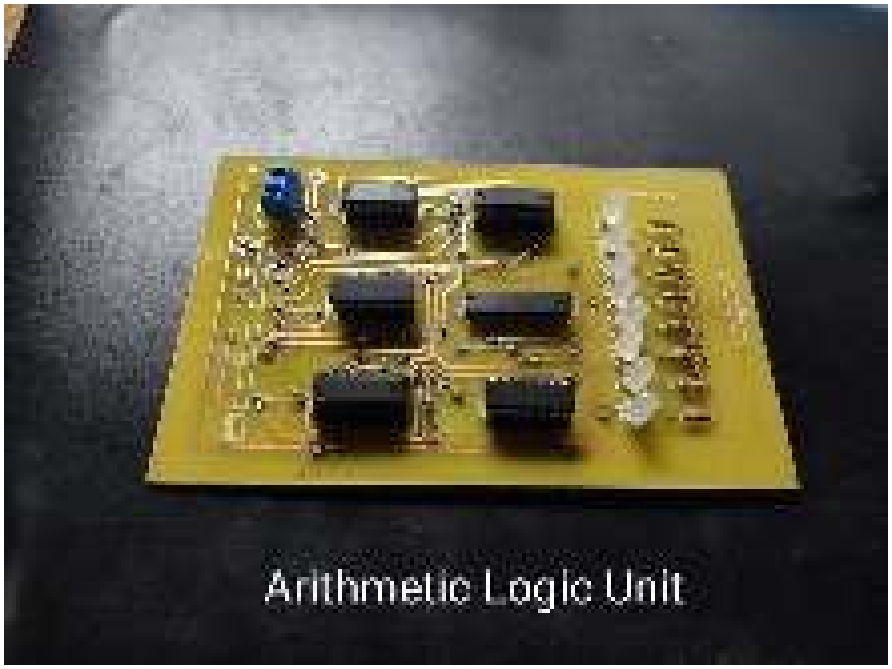
16.3 Accumulator / Register A



PCB — Accumulator / Register A

8-bit register holding one ALU operand and storing the result. Permanent internal connection to the ALU. AI loads from bus; AO drives bus via tri-state buffer.

16.4 Arithmetic Logic Unit (ALU)



PCB — Arithmetic Logic Unit (ALU)

8-bit addition (SU=0) and subtraction (SU=1) using 2's complement. EO enables the ALU result onto the bus; disconnected (Hi-Z) when EO is low.

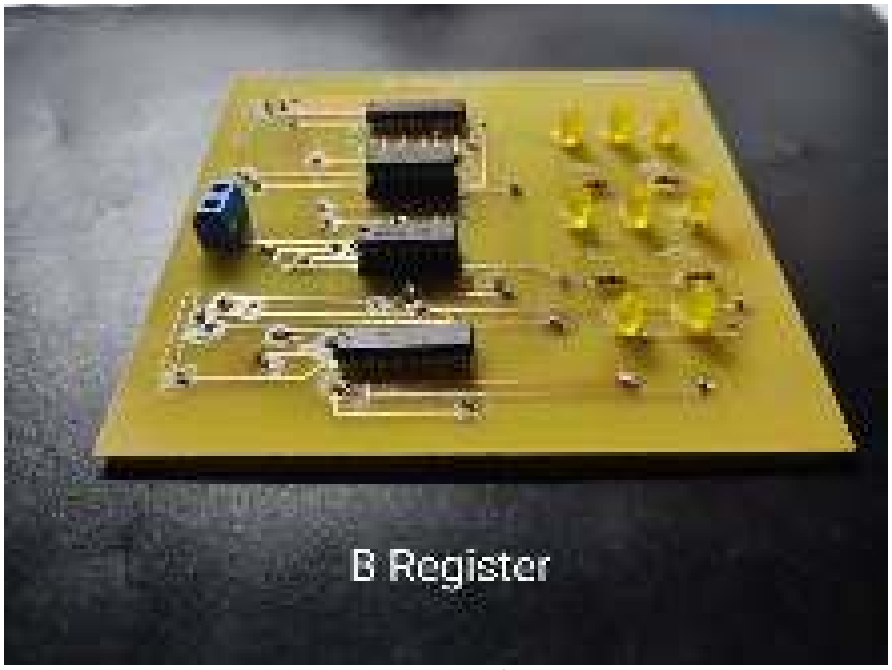
16.5 RAM / MAR / Input Unit / Address Selector



PCB — RAM / MAR / Input Unit / Address Selector

Combined PCB: 16×8-bit RAM, 4-bit MAR (74HC173), 8 DATA + 4 ADRS input switches, and Address Selector (logic gates). RUN mode → MAR addresses RAM; PROG mode → switches address RAM.

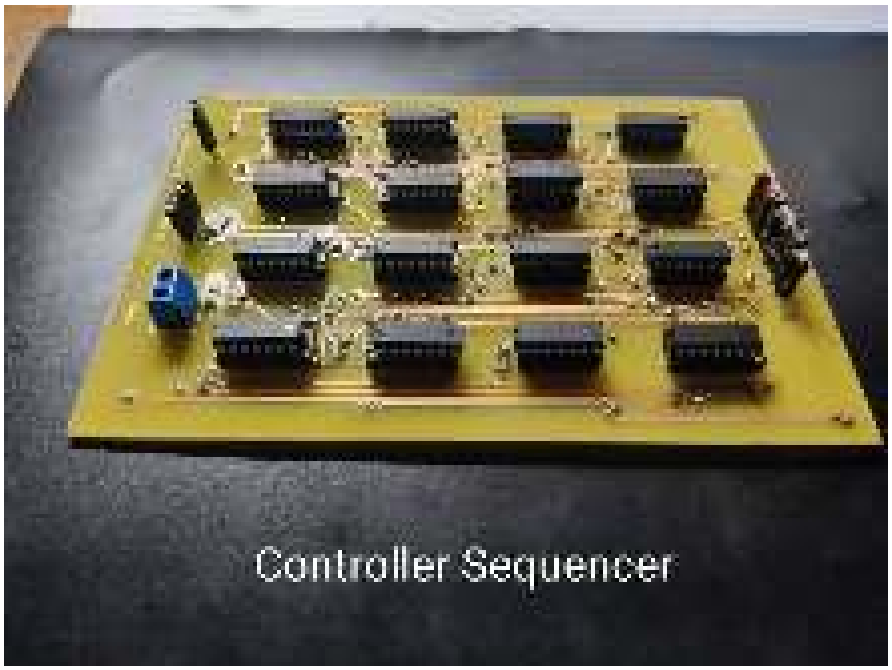
16.6 B Register



PCB — B Register

8-bit register for the ALU's second operand via permanent internal connection. BI loads from bus; no bus output (write-only from bus perspective).

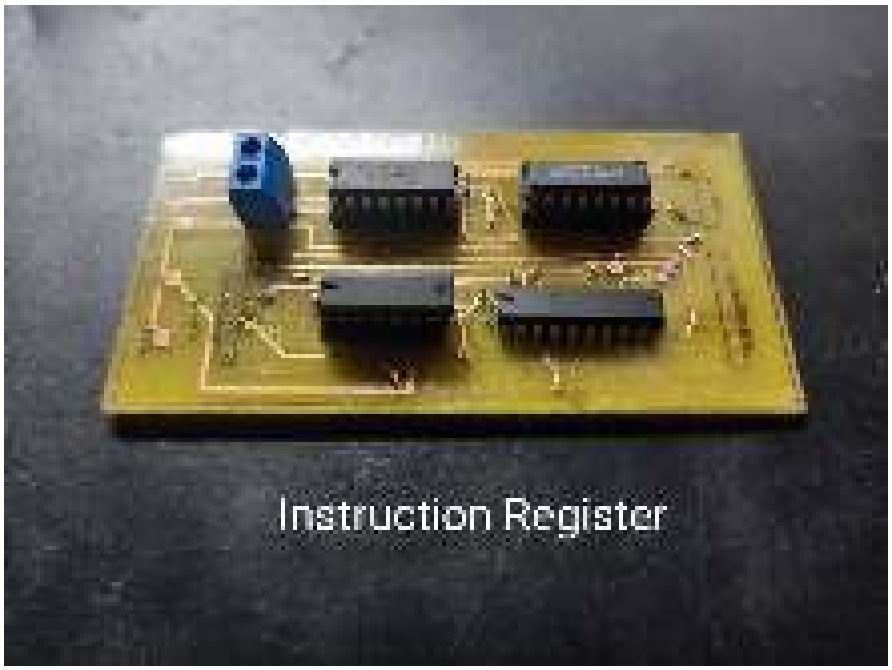
16.7 Controller / Sequencer



PCB — Controller / Sequencer

Generates all 12 control bits (CE CO MI RO II IO AI AO SU EO BI OI) + HLT each T state. Implemented with 5 T states (T2 and T3 merged) due to PCB manufacturing issues and time constraints.

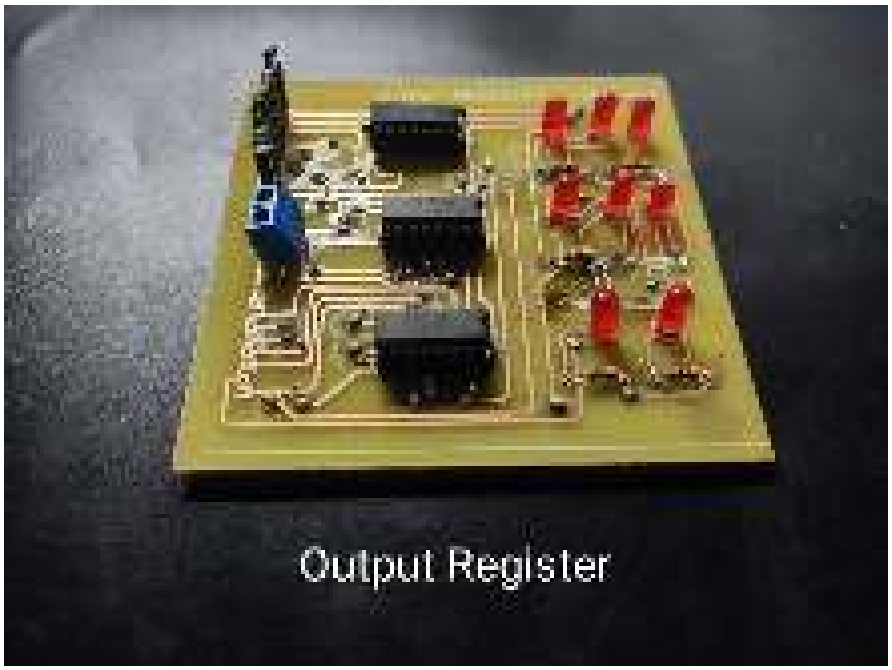
16.8 Instruction Register



PCB — Instruction Register

Stores the 8-bit fetched instruction. Upper nibble (opcode) feeds permanently to the Controller/Sequencer; lower nibble (address field) is driven onto the bus when IO is active.

16.9 Output Register



PCB — Output Register

8-bit register capturing the Accumulator's value on OUT instruction (OI active). Drives the binary LED display. 7-segment display was not implemented due to time constraints.